A PROJECT REPORT

ON

LOCALMIND
LOCAL-FIRST AGENTIC KNOWLEDGE COPILOT WITH A CUSTOM MCP MEMORY SERVER

A REPORT SUBMITTED TOWARDS PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER SCIENCE ENGINEERING

SUBMITTED BY

Trijal Burman - 23070123145

Tiya Singh - 23070123144

Gayatri Rajesh Apotikar - 23070123055

Nimita Jestin - 23070122259

॥वसुधैव कुटुम्बकम्॥

UNDER THE GUIDANCE OF

Dr. Sonali Kothari

SYMBIOSIS INSTITUTE OF TECHNOLOGY
PUNE - 412115

A CONSTITUENT OF
SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

ACADEMIC YEAR 23-27

Declaration

We declare that the project entitled “LocalMind: Local-First Agentic Knowledge Copilot with a Custom MCP Memory Server”, submitted to Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune, is our project work for the Bachelor of Computer Science Engineering programme. We are responsible for the claims, sources, evaluation records and final submission contained in this report.

Prior research and software sources are acknowledged in the references. AI assistance in software development, experimental support and report drafting is disclosed in Appendix C. We will review this declaration against institutional requirements before signing. No faculty approval, publication or plagiarism score is implied by this unsigned copy.

Under the guidance of Dr. Sonali Kothari. Department: Computer Science Engineering. Academic year: 23-27.

Trijal Burman
23070123145
Signature: _____

Tiya Singh
23070123144
Signature: _____

Gayatri Rajesh Apotikar
23070123055
Signature: _____

Nimita Jestin
23070122259
Signature: _____

Date: _____

This is to certify that the project work entitled “LocalMind: Local-First Agentic Knowledge Copilot with a Custom MCP Memory Server” has been prepared by Trijal Burman, Tiya Singh, Gayatri Rajesh Apotikar and Nimita Jestin, in partial fulfillment of the requirements of the Bachelor of Computer Science Engineering programme, at Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune, for the academic year 23–27.

Faculty certification and approval are to be completed after institutional review.

Dr. Sonali Kothari
Project Guide
Signature: _____

Acknowledgement

We express our sincere gratitude to our project guide, Dr. Sonali Kothari, for her guidance and supervision of LocalMind. Her support helped us frame the project around practical local knowledge management and the need to evaluate an AI system beyond the fluency of its responses.

We thank Symbiosis Institute of Technology, Pune, for the opportunity to undertake this project as part of the Bachelor of Computer Science Engineering programme. We acknowledge the researchers and maintainers whose openly available work on retrieval, local inference, SQLite and the Model Context Protocol made the implementation and study possible.

We also thank our families, classmates and friends for their support. We acknowledge the AI-assisted development and writing process and retain responsibility for checking every submitted claim. In particular, we preserve the unsuccessful experimental outcomes because they are essential to understanding the system and improving it responsibly.

Trijal Burman

Tiya Singh

Gayatri Rajesh Apotikar

Nimita Jestin

List of Figures

Fig. No.	Title	Page
1	Local processing and MCP architecture	20
2	Library view in the isolated local rehearsal	21
3	Persistent source, memory and conversation entities	22
4	Historical evidence loss and earlier research diagnostic	28

List of Tables

Table No.	Title	Page
1	Objective and verification mapping	11
2	Principal software components	16
3	Evaluated laptop configuration	16
4	Main work packages	17
5	Cost categories and estimation basis	19
6	Retrieval on 96 synthetic queries	27
7	Historical generation outcomes before repair	28
8	Historical baseline and production repair outcomes	29
9	Requested facts retained in final summaries	29
10	Historical resource and runtime context	30
11	Historical reproduction settings	36
12	Demonstration checks and recovery	37
13	Mandatory attachment register	37
14	Project evidence and preservation	38

Abstract

LocalMind is a local-first knowledge copilot that combines approved-folder indexing, persistent project memory, local language-model inference and a custom Model Context Protocol (MCP) server. The system uses a browser interface, FastAPI, SQLite FTS5, dense cosine retrieval, reciprocal rank fusion and Ollama. It supports source inspection, conversational assistance, document summarization and saved project plans without mandatory paid inference APIs after installation.

Evaluation on LM-Synth v1 uses 39 markdown files, 141 chunks and 96 retrieval questions. Historical lexical and hybrid retrieval achieve complete evidence coverage at five candidates on 96/96 questions; dense retrieval achieves 91/96. The original generation paths answer 0/24 positive questions completely after losing the reference block. Following an evaluation-only diagnostic, the production fitter was repaired to measure actual serialized cost. A direct replay retains reference evidence in 64/64 requests and yields 7/24 complete answers for RAG and 3/24 for the MCP agent. Historical summaries retain 10/24 requested facts and were not rerun.

Latest checks pass 163 Linux tests and 200 tests for the Windows copy on Linux, with six native Windows checks skipped. Greetings bypass retrieval and inference; filename checks use indexed metadata, and source cards group repeated passages without renumbering citations. The historical replay reuses inspected questions and is not independent accuracy confirmation. Local integration is feasible on the Ryzen 7 250 / RTX 5050 laptop, while semantic correctness and broader reliability remain unestablished.

Keywords: local-first AI; MCP; persistent memory; agentic RAG; SQLite; evidence delivery.

Contents

Declaration 2
Acknowledgement 3
List of Figures 4
List of Tables 5
Abstract 6
List of Abbreviations 9
1. Introduction **10**
1.1 Introduction 10
1.2 Problem Statement 10
1.3 Scope of Research 10
1.4 Research Hypothesis 11
1.5 Objectives 11
1.6 Overview of the Report 12
2. Literature Review **12**
2.1 Retrieval-Augmented Generation and Dense Representations 12
2.2 Lexical Retrieval and Reciprocal Rank Fusion 13
2.3 Tool-Using Agents and Bounded Actions 13
2.4 Persistent Memory Architectures 13
2.5 Local Ownership, MCP, and Untrusted Documents 14
2.6 Evaluation: Evidence, Context, and Citations 14
2.7 Research Gap and LocalMind’s Positioning 15
3. Platforms Tools and Frameworks Used **15**
3.1 Software Tools and Frameworks 15
3.2 Hardware and Execution Environment 16
4. Software Requirements Specification **17**
4.1 Work Breakdown Structure 17
4.2 Functional Requirements: Sources and Retrieval 17
4.3 Functional Requirements: Memory and Assistance 18
4.4 Non-Functional Requirements 18
4.5 Project Cost Estimation 18
5. System Design **19**
5.1 System Architecture 19

Contents continued

5.2 User Interface and Data Flow 20
5.3 Persistent Data Model 21
5.4 Transactions and Retrieval State 22
6. Implementation 23
6.1 Approving Sources and Restricting Scope 23
6.2 Bounded Reading and Document Extraction 23
6.3 Page-Aware Chunking and Embeddings 23
6.4 Lexical, Semantic and Hybrid Ranking 24
6.5 Incremental Indexing and Source Freshness 24
6.6 MCP Server and Client Execution 25
6.7 Conversation Context and Bounded Agency 25
6.8 Whole-Document Summaries and Project Plans 26
6.9 Privacy Controls and Evaluation Protocol 26
7. Results and Discussion 26
7.1 Evaluation Overview 26
7.2 Retrieval Evidence Coverage 27
7.3 Historical Factual-Answer Baseline 27
7.4 Evidence Delivery and Context Audit 28
7.5 Production Context Repair and Replay 29
7.6 Long-Document Summarization 29
7.7 Functional and Runtime Observations 30
7.8 Validity Threats and Error Analysis 30
8. Conclusion and Future Scope 31
8.1 Conclusion 31
8.2 Future Scope 31
9. References 33
Appendices 36
A. Reproducibility record 36
B. Demonstration acceptance cases 36
C. Academic integrity and required reports 37
D. Submission and evidence checklist 37

List of Abbreviations

Term	Meaning
AFVM	All-fact value match
API	Application programming interface
BM25	Best Matching 25 ranking family
CES@k	Complete evidence success among top k results
CI	Continuous integration / confidence interval (context dependent)
FTS5	SQLite full-text search extension
FVM	Fact-value match
LLM	Large language model
MCP	Model Context Protocol
MRR	Mean reciprocal rank
OCR	Optical character recognition
RAG	Retrieval-augmented generation
RRF	Reciprocal rank fusion
RSS	Resident set size
VRAM	Video random-access memory

1. Introduction

1.1 Introduction

LocalMind is a local-first knowledge copilot for files, project notes and deliberately saved memories. It combines a browser interface, a local language model, document retrieval and a custom Model Context Protocol (MCP) server. The intended user is a student or knowledge worker whose useful context is distributed across folders rather than one searchable project workspace.

Users connect selected folders, inspect extracted text, ask questions and save facts they want to retain. Originals remain at their existing paths; the application stores its index and conversational state locally. A new conversation can use persistent project memory without depending on unlimited chat history. These operations remain subject to approved-source, format and resource limits.

This report distinguishes the historical evidence-delivery failure, an evaluation-only diagnostic and a recorded production repair replay. Later implementation work isolates greetings, checks exact filenames through MCP and fits complete source records carrying document identities. The browser groups repeated passages without changing their citation numbers. These improvements support inspection and regression testing; they do not establish general answer accuracy or introduce a new retrieval algorithm.

1.2 Problem Statement

A project decision may be recorded in a Markdown note, an implementation detail in source code and a schedule in a PDF. Filename search helps when the user remembers suitable words. A conversational assistant without access to those records may produce a plausible answer from model parameters or require repeated uploads. Persistent local context addresses this practical fragmentation.

The engineering problem includes obtaining suitable evidence, identifying its exact source and delivering it intact to the model. Historical retrieval found expected material, but approximate clipping discarded the evidence message after JSON escaping expanded it. Exact serialized UTF-8 accounting repairs that failure. The current implementation also separates social turns and exact catalogue questions from generation, while retaining limits in evidence selection and semantic verification.

The research question is how effectively a compact local model can use approved evidence through a custom MCP service on a constrained laptop. Retrieval, context retention, alias-based factual completion, summaries and resource demand are evaluated separately. The new production replay reuses frozen questions and is not held-out confirmation. Privacy means local processing after installation; zero recurring inference-API charges do not eliminate hardware, electricity, storage or maintenance costs.

1.3 Scope of Research

The system covers approved folders containing supported text, Markdown, code, DOCX text and PDFs with extractable text. It provides indexing, hybrid search, extraction viewing, source-linked chat, summaries, explicit memories, project workspaces and saved plans. The Windows edition supplies platform-specific setup and filesystem safeguards. Context and grounding repairs are already versioned; subsequent routing and display updates have separate regression records.

Local drive means files already available on the laptop HDD, SSD or an explicitly approved accessible path. The application does not silently scan every partition. Arbitrary binary understanding, image-only PDF OCR, continuous watching, encrypted storage and autonomous code execution remain outside scope. Exact message fitting preserves bounded evidence input; it does not guarantee inclusion of every fact or correct generation.

1.4 Research Hypothesis

H1 predicts complete expected evidence within five results on the synthetic corpus; this holds historically for lexical and hybrid retrieval. H2 predicts reliable complete factual answers; the recorded repair replay reaches 7/24 and 3/24, leaving reliability unsupported. H3 predicts an adaptive MCP benefit; no additional model-selected tool executes in that replay. Later small model-selection probes reuse inspected examples and do not independently confirm these hypotheses.

1.5 Objectives

The primary objective is an inspectable local workflow: approve a folder, retrieve content, verify a source, retain a selected fact and reuse project context. A second objective is genuine MCP communication between the agent and persistent knowledge operations. A third is to preserve evidence across context fitting and evaluate failures with reproducible inputs.

The original experiment, diagnostic and 64-request production repair replay remain distinct historical records. Later regression checks exercise greeting isolation, exact metadata lookup, source identity, citation bounds and forced final synthesis. Passing these checks establishes the mechanisms that were tested. It does not prove semantic entailment, complete answers on arbitrary documents or useful autonomous tool selection.

Objective	**Verification evidence**
Local document access	Approved-root and extraction functional tests
Persistent memory	SQLite storage and project-scope tests
Protocol-based tools	Official SDK stdio MCP round trips
Retrieval quality	96 queries; evidence coverage at k
Answer reliability	Historical baseline and production replay; 24 positives + 8 unknowns
Platform support	Current Linux checks; historical native Windows CI

Table 1: Objective and verification mapping

The acceptance evidence is intentionally mixed. Passing integration tests establishes that operations execute as specified; it does not prove that generated prose is correct. The report therefore preserves both the functioning infrastructure and the failed factual-answer experiment, and uses the latter to prioritize future engineering work.

1.6 Overview of the Report

Chapter 2 reviews retrieval-augmented generation, sparse and dense search, tool use, memory and local-first software. It distinguishes established methods from integration choices and explains why retrieval, evidence delivery and final factual quality require separate evaluation.

Chapter 3 presents the software platforms, tools, frameworks and target hardware. Chapter 4 specifies the work breakdown, functional and non-functional requirements and cost basis. Chapter 5 describes the system architecture, interface data flow, persistent data model and transactional retrieval state. Chapter 6 follows folder approval, extraction, ranking, MCP communication, memory, model context and summarization, including the applied context-fitting repair and remaining limitations.

Chapter 7 preserves the historical baseline, diagnostic and production repair replay, then distinguishes later functional checks and the small model-selection probe. Retrieval, summary and resource measurements retain their original scope. Chapter 8 records completed fixes and remaining work. Chapter 9 lists the references; the appendices preserve provenance, demonstration instructions and the status of institutional attachments.

The report follows the supplied sample layout, with preliminary material, nine numbered chapters and supporting appendices. The contents and caption lists use physical PDF page numbers. The companion IEEE paper and frozen research records retain their distinct experimental scope and provenance.

2. Literature Review

2.1 Retrieval-Augmented Generation and Dense Representations

Retrieval-augmented generation connects a language model to evidence outside its parameters. Lewis et al. [1] establish this combination as a way to use external knowledge during generation. For LocalMind, the practical implication is that current project files can supply context without retraining the chat model. The original research includes trained components and public tasks; its reported performance cannot be transferred to this different local pipeline.

Karpukhin et al. [2] investigate dense passage retrieval through learned question and passage encoders. Reimers and Gurevych [3] demonstrate efficient independent sentence representations for similarity comparison. These works explain the motivation for semantic retrieval when wording differs between a question and a document. LocalMind uses EmbeddingGemma instead of those encoders, so the cited studies provide foundations rather than an implementation identity or an expected accuracy figure.

The distinction matters for personal knowledge. Project names, identifiers, filenames, and technical terms may remain unchanged even when a question is paraphrased. Dense similarity should therefore be compared with exact lexical matching on the same extracted passages. Retrieval also precedes prompt construction: a correct search result is useful only if its evidence survives formatting and context limits. LocalMind consequently evaluates source coverage separately from final answer-value matching, preserving the possibility that retrieval succeeds while downstream generation fails. This staged view avoids treating a source card as proof of a grounded answer.

2.2 Lexical Retrieval and Reciprocal Rank Fusion

BM25 remains a useful lexical foundation because it combines term-frequency saturation with document-length normalization [4]. Exact terminology is common in notes, specifications, and code. LocalMind implements lexical search with SQLite FTS5 across chunk text, title, and path. Its field weights are 1.0, 2.0, and 0.3. SQLite negates the conventional BM25 expression, so better matches have numerically lower scores [5].

Reciprocal rank fusion combines positions from separate rankings rather than calibrating their raw score scales [6]. LocalMind adds equal contributions of 1/(60 + rank) from lexical and dense lists, with absent candidates contributing zero. This is an established fusion rule, not a new algorithm. The dense component uses positive cosine similarity over compatible local embeddings. If embeddings are unavailable, lexical search remains usable without hosted services.

The combination is plausible but must be tested against both components. A dense result can improve recall while lowering the position of an exact lexical match; fusion therefore need not dominate every query. On the small identifier-rich LocalMind corpus, lexical and hybrid methods both reach complete-evidence success for 96 of 96 questions within five results. Hybrid adds no gain at that cutoff. These conditional findings support retaining a strong baseline rather than presuming that a more complex retrieval method is universally better. They do not establish answer accuracy or large-archive performance.

2.3 Tool-Using Agents and Bounded Actions

A conversational assistant becomes more useful when it can obtain missing information through defined operations. ReAct studies interaction between reasoning and actions [7], while Toolformer investigates learning when and how to call APIs [8]. LocalMind draws on this general tool-use direction. It uses an existing pretrained model and runtime checks; it does not reproduce Toolformer's training method or experimentally expose internal reasoning.

The implemented agent receives seven read-only schemas, including exact filename lookup alongside search, reading, recall, document listing, project listing and conversation lookup. Ordinary generation permits four planning rounds and six additional tool executions after initial search and recall. A final synthesis disables tools and repacks original evidence; one citation-repair retry may follow. The host allowlist prevents these model requests from becoming arbitrary shell or filesystem actions.

An available tool is not evidence that the model used it effectively. The historical runs and recorded repair replay execute no additional model-selected tools. Later changes improve routing, whole-record evidence fitting and reference validation without demonstrating an adaptive retrieval advantage. Valid citation numbers identify supplied records; they do not prove a claim follows from their text. Future experiments need tasks requiring additional reads and independent assessment of the resulting answers.

2.4 Persistent Memory Architectures

Persistent memory is broader than saving a chat transcript. MemGPT explores hierarchical context and virtual memory management [9]. MemoryBank considers user-oriented memory and updating or forgetting information [10]. LongMem augments language modeling through a learned memory

architecture [11]. These designs illustrate different mechanisms; their benefits cannot be assumed for a system that simply stores relational records.

LocalMind keeps explicit notes, project descriptions, sessions, and conversation messages in SQLite. Notes can have a kind and an optional project association. Memory lookup uses bounded substring filtering, while conversation search uses FTS5. The implementation has no knowledge graph, learned consolidation, automatic preference extraction, or forgetting schedule. A record surviving a restart establishes persistence, but does not guarantee that it will be retrieved or fit into a later prompt. This separation is central to an accurate account of personalization.

Related agent research adds further distinctions. Generative Agents includes memory and reflection for simulated behavior [12], while Reflexion retains textual feedback across trials [13]. LocalMind does not implement those reflection loops. Its user explicitly controls memory writes, and retrieved text cannot directly invoke the current-user memory command path. This restriction keeps stored knowledge inspectable and avoids presenting ordinary conversation history as autonomous learning. Future evaluation should measure whether saved context improves later assistance, including stale preferences and conflicting notes, rather than equating database presence with useful long-term memory.

2.5 Local Ownership, MCP, and Untrusted Documents

Local-first software emphasizes ownership, offline access, and user agency, alongside a broader vision of collaborative replication [14]. LocalMind implements single-laptop processing rather than replicated collaboration. Its application, database, approved folders, and model server can operate locally after installation. Model and dependency downloads need Internet access initially. Avoiding paid inference APIs therefore removes an API fee, not hardware, storage, electricity, installation bandwidth, or maintenance costs.

MCP supplies an interoperable host, client, and server interface for tools and resources [15]. LocalMind negotiates an actual SDK session with a separate stdio server and exchanges structured results. The server exposes both reads and explicit mutations, but the built-in model receives only read tools. Protocol annotations describe behavior; they do not enforce authorization. External clients need their own controls and can transmit returned content outside the local boundary.

Privacy and security must remain separate claims. Indirect prompt injection can embed instructions in retrieved material [16]. Local inference does not make such content trustworthy. LocalMind marks it as reference data and limits model capabilities, yet reports no adversarial security proof. The database is unencrypted, and software with the user's permissions can read it. Folder approval also bounds access: the application does not silently scan the entire drive. These choices support a controlled workflow without establishing confidentiality against compromised local software or every possible document instruction.

2.6 Evaluation: Evidence, Context, and Citations

BEIR emphasizes heterogeneous retrieval tasks and the continuing value of lexical baselines [17]. MTEB evaluates embeddings across varied tasks and languages [18]. These references motivate broader future testing; their scores are not LocalMind results. A small English corpus with recurring project

identifiers cannot establish general personal-drive performance. Evaluation needs an explicit corpus, frozen questions, correct source paths, and clearly defined success criteria.

RAGAs separates retrieval, faithfulness, and answer quality [19]. Citation research further asks whether a reference supports a generated claim [20]. LocalMind consequently distinguishes complete-evidence success, fact-value matching, source-index validity, and gold-path coverage. A correct citation index can accompany a wrong date or incomplete answer. Its deterministic alias scorer also has limits: a negated or misattributed value can match, and a valid unlisted paraphrase can fail. No language model judges the reported quantitative results.

Lost in the Middle shows that long context does not guarantee uniform use of relevant information [21]. The historical LocalMind summary experiment likewise separates section visitation from final value retention: fifteen sections were processed, but only ten of twenty-four requested values survived. This does not reproduce the earlier study's mechanism. The production context repair replay did not repeat those summary tasks. Retrieval, prompt delivery and output claims therefore require separate records; a completed serialization repair should not be reported as a demonstrated solution to long-document summarization.

2.7 Research Gap and LocalMind's Positioning

Prior work establishes retrieval-augmented generation, sparse and dense matching, fusion, tool use, memory and local ownership [1], [6], [7], [14]. LocalMind addresses practical integration on a constrained laptop, not an absence of earlier memory assistants. It combines approved-folder access, persistent context and an inspectable MCP interface.

The engineering contribution includes exact serialized UTF-8 fitting, complete source records, local serving, SQLite retrieval and paginated extraction. Exact filename questions now use metadata tools rather than inference from search snippets. No learned retrieval or memory algorithm is introduced. Persistence still differs from useful recall, and valid source references differ from supported claims [9], [19], [20].

Historical lexical and hybrid retrieval cover all 96 questions within five results. Both original generators complete 0/24 positives after evidence loss. The separate repaired replay retains references for 64 requests and reaches 7/24 single-pass and 3/24 agent AFVM. Later functional regressions and model-selection probes address implementation defects but do not replace these frozen findings. Heterogeneous archives, summary retention and semantic assessment remain open work.

3. Platforms Tools and Frameworks Used

3.1 Software Tools and Frameworks

The software requirements translate the need for private, persistent assistance into components that can run on a personal laptop. LocalMind combines a browser interface, a local HTTP application, document processing, durable storage and an independent MCP memory server. The browser presents approved folders, conversations, memories and projects. The application coordinates these functions without requiring a paid inference service.

Python provides the common implementation language. FastAPI and Uvicorn serve the local interface [22], [23], while the official MCP SDK handles protocol initialization, tool discovery and tool execution. SQLite stores both structured records and searchable text. NumPy supports vector calculations, and pypdf extracts textual PDF content. DOCX processing uses Python ZIP and XML libraries. Ollama supplies installed local language and embedding models [24]. Its local runtime configuration is described in the official FAQ [25].

Component	**Purpose**
FastAPI / Uvicorn	Local browser application
SQLite / FTS5	Durable records and lexical search
MCP SDK	Separate stdio memory service
Ollama / NumPy	Local inference and vector scoring
pypdf / ZIP / XML	Supported document extraction

Table 2: Principal software components

3.2 Hardware and Execution Environment

The target configuration is an existing laptop with an AMD Ryzen 7 250 processor, an NVIDIA RTX 5050 Laptop GPU and nominal 16 GB system memory. This configuration motivates a compact quantized language model and a local embedding model. It also motivates explicit limits on document size, extraction, context length and tool execution. Available memory must support the desktop, model runtime, application and temporary processing together.

The measured Linux environment exposed approximately 14,768.9 MiB system memory and 8,151 MiB GPU memory. Source defaults remain Qwen3 4B Q4_K_M with an 8,192 context setting. The Qwen3 family is described in its technical report [26]. An optional tested profile uses Qwen3.5 9B Q4_K_M at 12,288 with partial CPU offload; it is not a universal sizing guarantee. EmbeddingGemma supplies 768-dimensional vectors in this deployment; its model architecture and text representation design are documented in its technical report [27].

Item	**Observed configuration**
Processor	Ryzen 7 250; 8 cores, 16 threads
Graphics	RTX 5050 Laptop; 8,151 MiB VRAM
System memory	Nominal 16 GB; 14,768.9 MiB visible
Model runtime	Ollama 0.35.0
Operating system	Linux 7.0.0-38-generic

Table 3: Evaluated laptop configuration

4. Software Requirements Specification

4.1 Work Breakdown Structure

The work breakdown begins with scope definition: approved local files, supported document types, persistent memory and a browser interface suitable for a nontechnical user. Architectural work then establishes the boundaries between the API, indexing library, SQLite store, Ollama adapter and MCP process. Defining these boundaries early allows persistence and protocol behavior to be checked independently of model output.

Implementation proceeds through source validation, bounded readers, parsers and chunking before retrieval and generation. Storage work supplies transactional updates, lexical indexes and project relationships. The MCP server exposes these capabilities through explicit tools. The agent then consumes those tools, while the interface provides source approval, progress information, conversation history and memory management. Platform launchers are a separate deployment task.

Package	**Concrete output**
Requirements and design	Scope, component boundaries, schema
Knowledge processing	Safe extraction and incremental index
Memory and agent	MCP tools and bounded execution
Application and deployment	Browser interface and local launchers
Validation and reporting	Tests, benchmark records and analysis

Table 4: Main work packages

4.2 Functional Requirements: Sources and Retrieval

The first functional requirement is explicit source approval. The user selects an existing folder on a local disk, and the application records that scope before indexing. The scanner must reject protected or overlapping roots and avoid symbolic links, excluded directories and known secret filenames. Removing an approved source must remove its associated indexed documents through the stored ownership relationship.

The second requirement is useful extraction from supported files. Text, Markdown and code are decoded as text; PDFs and DOCX files are processed through their supported extraction paths. Successful extraction produces a complete stored text snapshot and bounded passages with exact character offsets. A rescan must update changed files, preserve valid unchanged work and remove absent documents only after a complete traversal.

The third requirement is retrieval with inspectable provenance. A query may combine lexical and semantic candidates, but every returned passage must identify its document and location metadata. Before delivery, the application checks that the source still belongs to an approved root and matches its indexed size and modification time. Document browsing and paginated extraction reads allow the user to inspect evidence beyond a short answer.

4.3 Functional Requirements: Memory and Assistance

Persistent assistance requires more than document search. Users must be able to create, edit and delete explicit memories, optionally associate them with projects and retrieve them in later conversations. A memory edit retains its identifier and original creation time. Sessions and messages persist conversation content, citations and execution traces, allowing relevant user and assistant messages to be searched across saved chats.

Complete greetings and capability-help turns use local responses without consulting documents, history or models. Exact filename and inventory questions use MCP metadata tools. Ordinary content questions retrieve evidence before generation and can fall back to exact excerpts. Additional read tools, whole-extraction summaries and editable project plans remain bounded application features. A greeting followed by a substantive request stays in the relevant document workflow.

Projects organize memories and conversations; they do not create separate filesystem permissions. Planning writes one PLAN.md rather than generating or executing a repository. Catalogue answers concern indexed approved sources, with counts and pagination; an absent match does not establish absence from the entire disk. Functional tool availability remains separate from effective model use and measured answer quality.

4.4 Non-Functional Requirements

Privacy is a deployment property supported by explicit technical boundaries. In the normal local interface, document extraction, embeddings, inference and persistence remain on the laptop. Ollama endpoints are limited to HTTP loopback addresses, redirects are disabled and proxy environment variables are ignored by the adapter. Downloading dependencies or models is an initial network activity and must be distinguished from later document processing.

Reliability requires durable transactions and visible failure behavior. SQLite foreign keys preserve ownership relationships, while write-ahead logging supports the API and MCP processes sharing one database. Failed document replacement rolls back instead of leaving partial chunks. Resource limits bound file reads, extracted text, scan candidates, passage size and agent execution. Embedding failure leaves lexical indexing available; unsupported or unreadable documents produce explicit errors.

Usability requires inspectable sources and editable memory. Cards group passages by document identifier, falling back to exact paths; unidentified records stay separate. Every original reference number remains on its card, and a twelve-card display cap is disclosed. Security still assumes a trusted local user: there is no application authentication, encrypted database or operating-system sandbox. Local execution alone is not complete security.

4.5 Project Cost Estimation

LocalMind avoids recurring charges to a hosted inference API by using installed open-source components and downloaded local models. The proposed deployment reuses the student laptop and its existing storage. Under that assumption, the additional hardware purchase cost can be zero. This statement concerns incremental project expenditure; it does not imply that computing hardware, electricity, internet access or development effort are free.

A transparent estimate separates fixed resources from variable operating costs. Storage must accommodate model weights, extracted document text, passage records, full-text indexes and vectors. Electricity cost can be calculated from measured average power, operating hours and the local tariff. Network expenditure depends on the initial downloads and future updates. No power measurement or market-price survey was conducted in the reported benchmark, so numerical estimates for these items would be unsupported.

Category	**Estimation basis**
Inference API	No paid hosted API required
Existing laptop	No additional purchase assumed
Electricity	kW × operating hours × local tariff
Storage / downloads	Actual capacity and connection usage
Maintenance	Student or user time; not measured

Table 5: Cost categories and estimation basis

5. System Design

5.1 System Architecture

The methodology separates deterministic knowledge handling from probabilistic language generation. Approved documents enter a controlled extraction and indexing pipeline. SQLite stores their text, metadata and vectors. A separate MCP server exposes retrieval and memory operations, while the agent uses those operations to assemble evidence for a local model. The browser interface provides the human point of control over approved sources and explicit memory.

Browser requests use loopback HTTP; agent knowledge operations use a separate MCP stdio process. Chat and embedding requests reach the locally installed Ollama runtime. Source originals remain in approved folders.

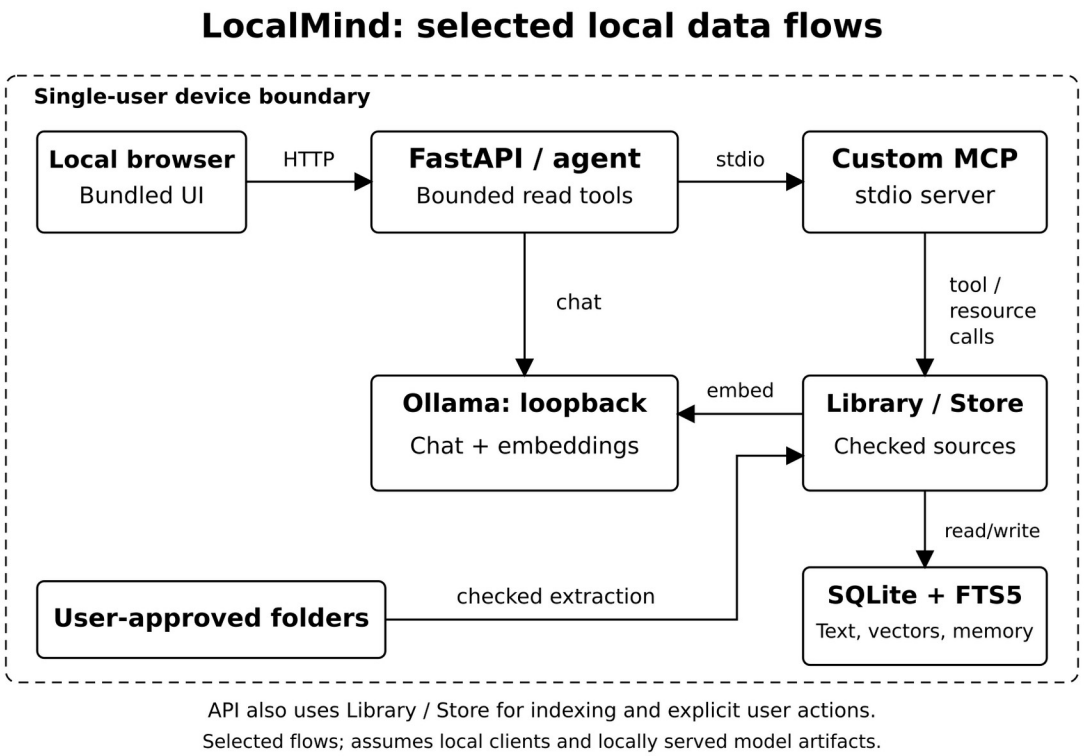


Fig 1. Local processing and MCP architecture

5.2 User Interface and Data Flow

The interface has five main areas: Copilot, Library, Memory, Projects and Settings. Library connects an explicitly chosen folder and opens extracted documents. Memory stores selected facts. Projects organize a workspace and its persistent notes; Settings confirms installed local models. Copilot combines a conversation with retrieved evidence and an inspectable tool trace.

The screenshot shows the earlier two-file Lighthouse rehearsal, not an accuracy test. Its model badge is historical. Current cards group repeated passages, retain every citation number and distinguish metadata-only file references from excerpts. Opening a card displays the corresponding indexed document. Source cards establish provenance, not semantic correctness.

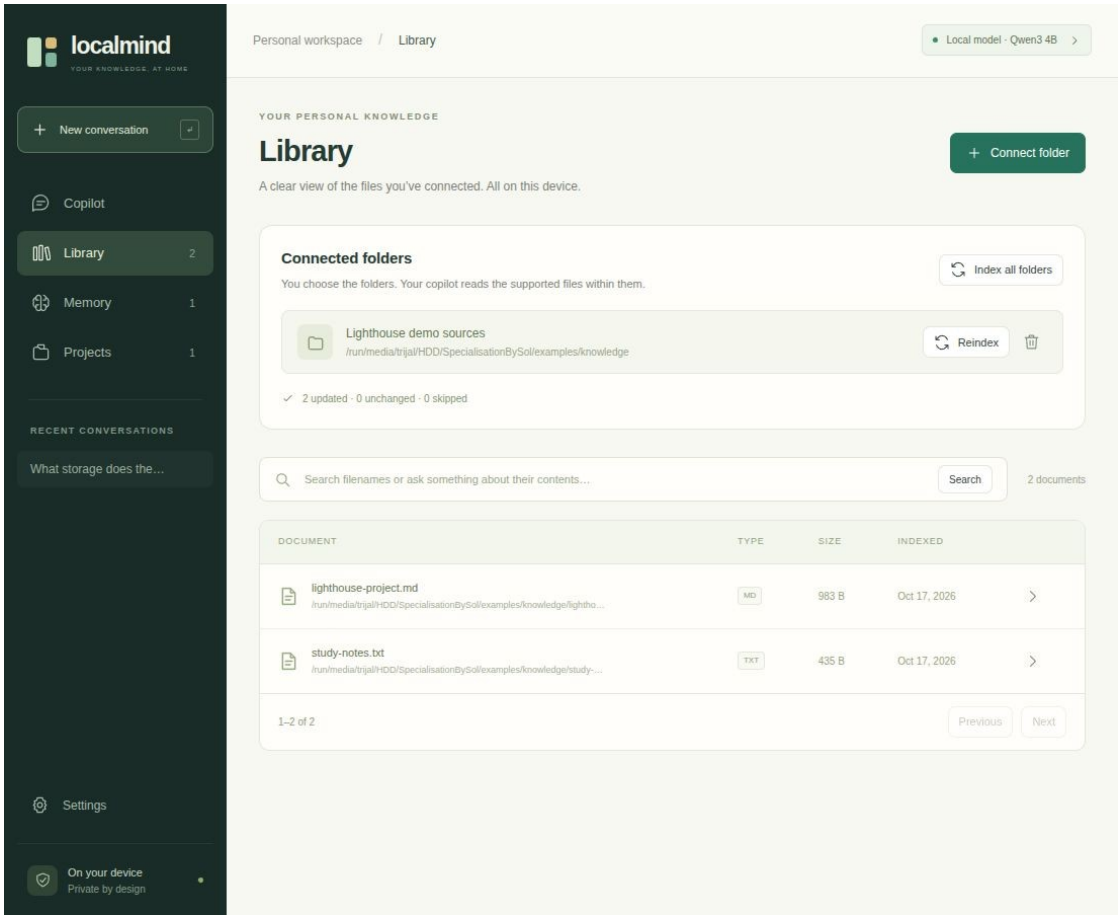


Fig 2. Library view in the isolated local rehearsal

5.3 Persistent Data Model

The database distinguishes source knowledge from personal working context. A root records an approved folder. Each document belongs to one root and stores its path, extracted text, extension, size, modification timestamp and indexing time. Each chunk belongs to a document and stores an exact text interval, an optional PDF page and an optional embedding with model and dimension metadata.

Memories and sessions may belong to a project; messages belong to sessions. Citation and trace JSON remain attached to messages. These are relational records, not an inferred knowledge graph.

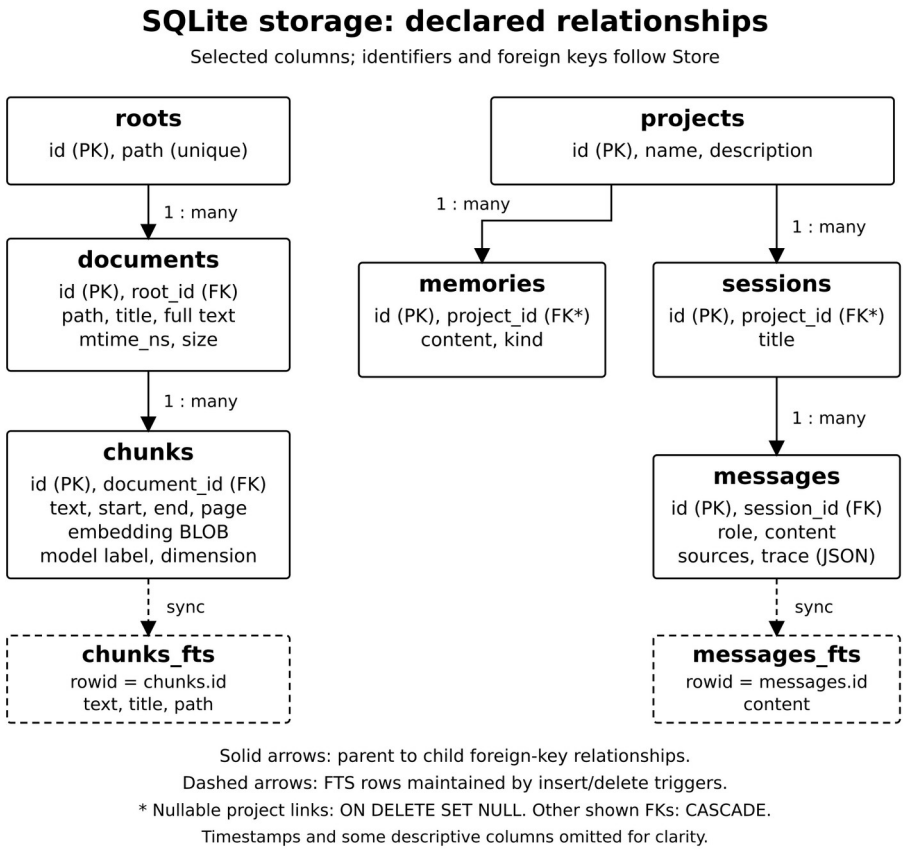


Fig 3. Persistent source, memory and conversation entities

5.4 Transactions and Retrieval State

Each store operation opens a short-lived SQLite connection with foreign keys enabled and a thirty-second busy timeout. Write-ahead logging allows the API and MCP processes to share a database without introducing an additional database service. SQLite documents concurrent readers and the single-writer behavior of WAL mode [28]. Writes remain transactional: replacing a document deletes its old chunks and inserts the new extraction and passages in one operation. An exception rolls back the replacement.

The full-text indexes use FTS5 with the unicode61 tokenizer. The chunk index contains text, document title and path; insert and delete triggers keep it aligned with chunk rows. Conversation search has a separate content index and includes user and assistant messages. Explicit memory search instead uses a bounded substring query over recent notes. A single phrase such as persistent memory therefore covers several deliberate mechanisms rather than one universal vector index.

Embeddings are stored as little-endian float32 binary values with their model label and dimension. Inserts reject nonfinite values and inconsistent dimensions within a document. Search compares only compatible vectors. This prevents an embedding-model change from silently mixing unrelated vector spaces. It does not eliminate the need to rescan or backfill embeddings when the configured model changes, nor does it establish semantic quality by itself.

6. Implementation

6.1 Approving Sources and Restricting Scope

Ingestion starts with an explicit folder choice. The library expands and canonicalizes the path, verifies that it is an existing directory and rejects prohibited locations. Overlapping approved roots are refused to avoid ambiguous ownership and duplicate indexing. The application data directory is excluded, preventing its own database, plans and runtime state from being ingested as ordinary source material.

The scanner does not follow symbolic links. It also omits hidden entries, dependency and build folders, system-related directories and known secret filenames. These exclusions reduce accidental collection but do not inspect the meaning of every document. A password written inside an otherwise allowed Markdown file remains possible. Source approval therefore grants access to eligible content inside that folder and should be treated as a meaningful user decision.

Platform handling follows the same scope goal through different filesystem APIs. Linux uses bounded descriptor-based reads with link avoidance. The Windows edition adds native checks for junctions, reparse points, network paths and handle destinations, while allowing ordinary folders on local fixed or removable drives. The approved scope is a document folder, not an unrestricted permission to read every system file on the laptop.

6.2 Bounded Reading and Document Extraction

A supported file is read within a configured twenty-mebibyte limit. The reader accepts regular files and compares file size and modification time before and after reading. If the source changes during ingestion, the operation fails and asks for a later rescan. The scanner also limits supported-file candidates to twenty thousand, while extraction limits each document to five million characters and PDFs to two thousand pages.

Text decoding recognizes Unicode byte-order marks, tries UTF-8 and falls back to CP1252 where appropriate. Binary-content checks reject common nontext inputs, and line endings are normalized. PDF extraction uses pypdf and records one-based page boundaries. The library documents text extraction and its distinction from OCR [29]. Scanned images without extractable text need a separate local OCR step; the application does not claim to understand them automatically.

DOCX extraction reads supported XML parts from the ZIP package, including paragraphs, table cells and available headers, footers, footnotes and endnotes. It preserves textual tabs and breaks without reproducing the original page layout or embedded graphics. The stored snapshot is therefore the complete successful extraction, not a visual or byte-identical copy. This distinction is necessary when describing full-document access and summarization.

6.3 Page-Aware Chunking and Embeddings

After extraction, the library divides text into passages while retaining exact start and end character offsets. The normal passage limit is 1,800 characters with 200 characters of overlap. When a passage would end before the document boundary, the algorithm prefers a newline or sentence delimiter in the

latter half of the candidate interval. This reduces abrupt splits without requiring an additional language-processing model.

PDF page intervals are processed separately, so one chunk never crosses two PDF pages. Empty passages are omitted, and a progress guard prevents overlap from causing an infinite loop. Character counts are intentionally described as characters rather than tokens because the local model tokenizer can produce different lengths for different content. Stored offsets allow passage text to be checked against the document snapshot exactly.

The local embedder processes batches of sixteen chunk texts. A document receives vectors only when all its embedding batches succeed with compatible finite dimensions. The first embedding failure disables further embedding attempts during that scan, allowing lexical indexing to continue. On a later rescan, unchanged documents can have missing or model-mismatched embeddings backfilled without reparsing their source. This fallback preserves access while making semantic retrieval availability explicit.

6.4 Lexical, Semantic and Hybrid Ranking

The query is trimmed to 1,024 characters and converted into at most twenty-four distinct Unicode alphanumeric tokens. Quoted tokens are joined with OR for FTS5. Lexical ordering uses native BM25 over passage text, title and path, with respective field weights of 1.0, 2.0 and 0.3. This stage is useful for exact project names, identifiers and terms found in local documents.

For semantic retrieval, the store scans compatible vectors in batches of 128 rows and computes cosine similarity. Vector normalization uses the NumPy norm primitive [30]. Invalid vectors, zero norms and nonpositive similarities are excluded. A bounded heap retains the strongest candidates. The implementation performs exact scanning rather than approximate nearest-neighbor search, so semantic work grows with the number of compatible vectors. No measured large-corpus scalability claim follows from this design.

Hybrid ordering applies equal-weight reciprocal-rank fusion: a candidate contributes $1/(60 + \text{rank})$ from each list in which it appears. Ranks start at one; an absent list contributes zero. Ties use chunk identifiers. The ranking depth is $\min(160, \max(40, 4L))$, where L is the requested store limit. Fusion scores are ordering values rather than probabilities. The library subsequently filters stale or disallowed sources before returning the requested number of passages.

6.5 Incremental Indexing and Source Freshness

A rescan compares each supported file with its stored size and nanosecond modification timestamp. Matching metadata allows the existing extraction to be reused, except when embeddings require backfilling. Changed documents are parsed and replaced transactionally. This avoids repeatedly processing every unchanged file while keeping source ownership, passage text and full-text entries consistent after a successful update.

Deletion handling depends on completing the folder traversal. The scanner collects seen paths and removes absent document rows only when enumeration finishes without traversal errors or the configured candidate limit. Otherwise pruning is suppressed. A failed parse can leave an older snapshot

in storage, but retrieval independently checks the live source signature and excludes it when metadata differs. The MCP document reader applies corresponding approval and freshness checks.

Freshness is deliberately limited to metadata validation. A content change that preserves both size and modification time can evade detection because the index does not use a cryptographic content hash. There is also no filesystem watcher; the user initiates rescans. These limits distinguish immediate rejection of a visibly stale snapshot from a guarantee that every stored byte always matches a changing disk.

6.6 MCP Server and Client Execution

The custom memory server uses the official SDK FastMCP implementation with stdio transport [31]. The application client starts a separate Python process, establishes a ClientSession, initializes the protocol, discovers tools and issues tool calls. Both processes use the configured SQLite database. This provides actual MCP interoperability without opening a network listener for the memory service.

Eleven tools cover knowledge search, paginated document listing, exact filename lookup, extraction reading, memory creation and recall, forgetting, projects, conversations and approved roots. Two resources expose overview and project information. Listing and lookup return totals, offsets and next offsets. Lookup matches exact basenames or paths, preserves duplicate names and reports indexed scope. Extraction reads default to 12,000 characters and allow at most 50,000 per call.

The built-in agent receives seven read-only schemas. External MCP clients may access additional write tools, and the browser can write memories directly. Tool annotations describe intent but are not authentication. Exact metadata responses keep filenames and paths from the store; generated text cannot rename a source. The MCP boundary is a capability interface for trusted local clients, not encrypted storage or permission to trust arbitrary clients.

6.7 Conversation Context and Bounded Agency

Complete social turns return locally before history, retrieval or model access. Exact inventory and filename requests use deterministic MCP metadata routes. Ordinary content questions use bounded recent context, initial knowledge_search and recall calls, five candidate passages and selected project or global memories. Earlier assistant claims are not reused as document evidence. Content requests mentioning a filename still reach the document workflow.

Ordinary generation exposes seven read-only schemas and allows at most eight actual MCP executions: two initial calls plus six additional tools. Four planning calls, one forced synthesis and at most one citation retry bound model calls at six. Forced synthesis removes tool schemas and repacks from original excerpts. Project recall and conversation search respect project scope; approved-file retrieval spans roots. Only the current user can trigger explicit remembering.

For context setting C, the fitter uses $B = C - \min(1536, \text{floor}(C/3)) - 256$ and exact serialized UTF-8 cost. Evidence packing retains whole numbered records containing text and source identity, using query-aware contiguous windows from 1,000 down to 60 characters. If an unusually long path cannot fit, that record is skipped while later candidates are tried. Displayed excerpts match delivered evidence; citation checks establish index syntax and bounds, not semantic entailment.

6.8 Whole-Document Summaries and Project Plans

Whole-document mode reads every page of the stored extraction through MCP pagination. At the default context setting, the agent divides text into sections of approximately four thousand characters and generates partial summaries. It then combines those partial outputs through iterative merging. This path differs from ordinary retrieval, where only a limited set of relevant passages is selected.

Processing every section establishes visitation, not complete factual retention. Intermediate inputs can be cropped and generation may omit or misstate facts. The historical summary experiment processed fifteen sections with eleven model calls per task and retained ten of twenty-four target facts. Those summary tasks were not repeated in the production question-answer repair replay. Summaries therefore remain editable assistance that should be checked against complete stored extracts.

Project planning is similarly bounded. The agent retrieves three passages, combines them with relevant project and global memories and generates a Markdown plan. The saved output is projects/<id>/PLAN.md beneath the application data directory. Regenerating the plan replaces that file. The feature helps organize objectives and next steps, but it does not scaffold an executable project, modify arbitrary user folders or run generated commands.

6.9 Privacy Controls and Evaluation Protocol

The privacy boundary includes approved folders, storage, API, MCP and the local runtime. Loopback validation, disabled redirects and proxy avoidance reduce accidental remote inference. HTTPX documents that trust_env=False disables environment-based client configuration [32]. A response guard checks cited references and can reject uncited actionable recommendations copied from untrusted evidence; bounded retry or exact excerpts provide recovery. These are limited controls. The application still lacks authentication, encrypted storage, an operating-system sandbox and comprehensive prompt-injection resistance.

Evaluation separates historical retrieval, original generation, a research-only diagnostic and the recorded production repair replay. The corpus contains 39 Markdown files and 141 chunks. Retrieval uses 96 positive questions; generation reuses 24 positives and eight unknowns per method. Later regression and model-selection records have distinct scopes. The historical summary and resource experiments were not rerun to establish current-model accuracy.

For CES@k, every expected fact needs an accepted alias in a retrieved chunk from its exact labeled source path among the first k results. AFVM requires all requested values in generated text; macro FVM averages positive-question coverage. Aliases and valid citation indices do not prove entailment. Latest functional checks pass 163 Linux and 200 Windows-copy tests on Linux, with six native checks skipped; the 64-request replay remains historical.

7. Results and Discussion

7.1 Evaluation Overview

The frozen LM-Synth v1 corpus contains 39 Markdown files, 201,518 extracted characters and 141 chunks. Twelve fictional project families supply 36 ordinary documents; three dossiers support

summary tasks. Retrieval evaluates 96 questions: 48 lexical, 36 paraphrase and 12 cross-document queries. Generation uses 24 selected positives and eight absent-value questions per method.

Three historical conditions remain distinct: the original snapshot loses evidence, an evaluation-only diagnostic tests the mechanism, and the recorded production repair replay verifies actual fitting without substitution. They reuse the same corpus and questions. Later routing, reference-identity and display changes have separate functional checks. Neither those checks nor a small postselected 9B probe replace the original measurements with held-out accuracy evidence.

Scoring uses accepted aliases and exact expected source paths. Complete evidence success requires all requested facts in matching source chunks. All-fact value match requires every requested value in an answer; macro FVM averages matched fractions per positive question. Neither proves semantic entailment or penalizes every contradiction. The small synthetic project families also limit independence and generalization. Historical retrieval, summary and resource results are labeled separately from the production repair replay.

7.2 Retrieval Evidence Coverage

The unchanged frozen retrieval experiment measures Complete Evidence Success at five candidates (CES@5). Each expected fact needs an accepted alias in a chunk from its exact labeled source path. Lexical and hybrid retrieval meet this criterion on all 96 queries; dense retrieval meets it on 91. Its five misses occur among paraphrase questions. These figures describe retrieval evidence, not final answers.

Retriever	**CES@5**	**MRR@10**
Lexical FTS5	96/96	0.9948
Dense cosine	91/96	0.8755
Hybrid RRF	96/96	0.9495

Table 6: Retrieval on 96 synthetic queries

Hybrid retrieval does not outperform lexical retrieval here. MRR uses the first chunk containing any expected fact, not all cross-document facts. Median production ranking takes 4.02 ms lexical and 4.76 ms hybrid, excluding query embedding (13.60 ms median). Dense control rankings are computed in memory and are not comparable service-latency measurements.

The result supports an inexpensive inspectable search layer for this workload. It does not establish performance on OCR, multilingual notes, large code repositories, ambiguous names or personal archives. Those require separate datasets and relevance judgments.

7.3 Historical Factual-Answer Baseline

Before the production repair, both single-pass RAG and the MCP agent matched no complete answers among 24 positive questions. Macro fact-value match was also zero. Each method returned the requested abstention phrase on all eight unknown questions and on every positive question. These are historical results of the original snapshot, not the current repaired application.

The requests completed without runtime exceptions, but their document-evidence messages did not survive context fitting. Source cards and successful MCP transport therefore coexisted with absent

model input. No additional model-selected tool call executed beyond the agent's initial search and recall. The baseline is preserved in its original result directory so that the later diagnostic and production repair cannot silently replace an unsuccessful outcome.

Historical condition	Complete positives	Unknown abstentions
Original single-pass RAG	0/24	8/8
Original MCP agent	0/24	8/8

Table 7: Historical generation outcomes before repair

The 24 positives are clustered within synthetic project families and do not establish a population failure rate for arbitrary drives. The engineering observation is narrower: the original evidence-delivery implementation failed on this sample. The separately recorded repair replay uses changed production code; later model-selection and regression results must retain their own scope.

7.4 Evidence Delivery and Context Audit

The historical audit found all 32 required fact slots in full returned passages for 24 positive questions. Keeping only the first 1,000 characters per passage reduced coverage to 15/32 slots and seven complete questions. Approximate context fitting then removed the entire evidence block, leaving zero slots. Fig 4 documents that earlier failure and research diagnostic.

The production fitter finds a complete-character prefix within actual serialized UTF-8 cost, accounting for escaping, metadata, shortening markers and schemas while pinning the system and current-user messages. The historical 64-request repair replay retained initial reference input within bounds. Later whole-record packing carries document identity and query-aware excerpts together. That later selection policy has not been evaluated by rerunning the frozen retrieval or summary experiment.

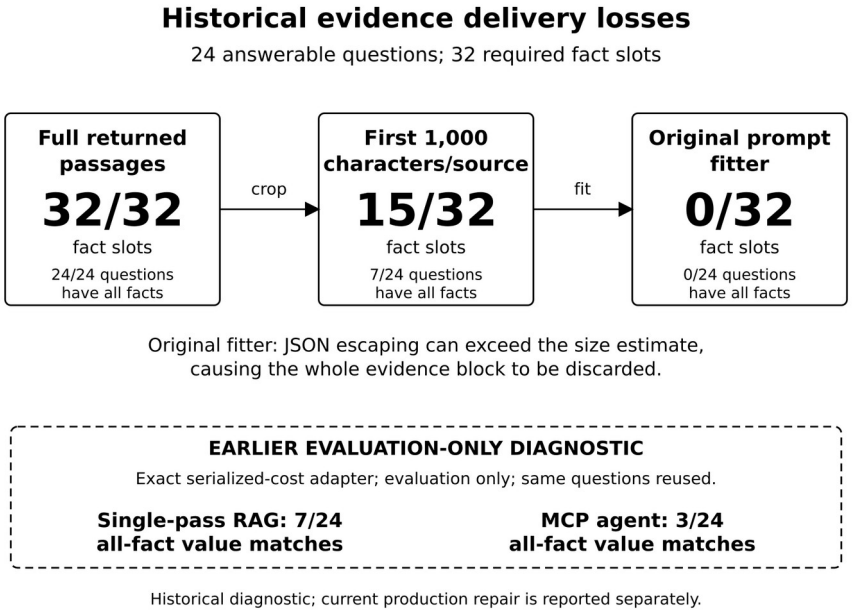


Fig 4. Historical evidence loss and earlier research diagnostic

7.5 Production Context Repair and Replay

The recorded repair replay exercises actual production fitting with real MCP calls and no diagnostic substitution. Both methods answer the same earlier 24 positives and eight unknowns. All 64 requests retain initial reference evidence, stay within the configured bound and finish without request errors. These are historical development observations using the 4B configuration, not results of the optional later 9B profile.

Condition	Complete positives	Macro FVM
Historical single-pass	0/24	0%
Repaired single-pass	7/24 (29.17%)	37.50%
Historical MCP agent	0/24	0%
Repaired MCP agent	3/24 (12.50%)	27.08%

Table 8: Historical baseline and production repair outcomes

Macro FVM averages fact coverage per positive question. The earlier diagnostic also produced 7/24 and 3/24; matching totals are not independent confirmation. Context and grounding repairs are now published in versioned branches. The original replay and source hashes remain frozen, while later greeting, catalogue and citation changes have separate checks. A release state must not be inferred from the old experiment alone.

In this historical replay, single-pass RAG uses the abstention phrase on 8/8 unknowns and the agent on 7/8. Each response uses one model call; neither method executes an additional model-selected tool. Current citation retries therefore cannot be credited for those earlier outcomes. Prefix coverage and semantic mistakes limit this evidence; it establishes improved delivery without demonstrating general reliability or agent advantage.

7.6 Long-Document Summarization

The historical summary experiment used a route that reads sections through MCP, summarizes individual sections and merges intermediate notes. All 15 expected sections across three dossiers were processed. This verifies traversal coverage, but the final summaries retained only 10 of the 24 requested fact values. Reading every section is therefore a necessary process property rather than a guarantee of complete output.

Dossier	Sections read	Values retained
Morrow Annex	5/5	3/8
Oriole Basin	5/5	7/8
Tamarisk Span	5/5	0/8
Total	15/15	10/24 (41.67%)

Table 9: Requested facts retained in final summaries

Each historical summary required 11 model calls and approximately 56.75–64.38 seconds. Some final outputs claim that facts are absent even though the source contains them. Such statements require source verification and must not be repeated as established document properties.

The production repair replay evaluated question answering, not these summary tasks; the earlier 10/24 result remains historical. Structured field extraction and source-linked checks before narrative merging remain future work. A summary should be presented as an editable draft and verified against the reader, even when every source section was visited.

7.7 Functional and Runtime Observations

Latest Linux validation passes 163 tests. The Windows copy tested on Linux passes 200, with six native checks skipped. Historical Windows Server 2025 CI passed 86 cases per interpreter on Python 3.12.10 and 3.13.15 with a fake runtime [33]. Those older native checks do not validate the latest changes or establish Windows 11 GPU inference performance.

The latest regressions exercise greeting isolation, exact file metadata, source identity, grouped passages, imperative-language checks, forced final repacking and oversized-path omission. Earlier 62/99 repair-test counts remain dated snapshots. One live hello took 0.0931 seconds; three follow-ups took 0.0305, 0.0314 and 0.0672 seconds. These local-route observations exclude model inference and are not general chatbot latency measurements.

Historical observation	**Measured value / qualification**
Indexing	39 documents / 141 chunks; 12.875 seconds
Process RSS peak sum	8,258.89 MiB; shared pages may double count
Total GPU peak	5,317 MiB; desktop included
Resource sampling	239 samples in original experiment

Table 10: Historical resource and runtime context

The historical repair replay median response times were 0.686 seconds for single-pass and 1.237 seconds for MCP. They mix complete and incomplete outputs on reused questions and are not a controlled speed comparison. The resource peaks above were not remeasured for later fixes or model selection. Energy use, archive scale and broader hardware performance remain unestablished.

7.8 Validity Threats and Error Analysis

Construct validity is limited by alias matching. An expected value can appear in a wrong explanation, while a valid unlisted paraphrase can be missed. The historical U003 date-role error illustrates why source presence and valid citation indices do not prove claim entailment. Repairing context fitting does not remove these semantic limitations.

Internal validity is limited by reused questions, schema overhead and unseeded generation at temperature 0.25. The recorded production replay follows an observed failure and diagnostic on the same examples. Later query-aware packing and citation retries change the implementation again. These development stages need distinct records and cannot be pooled as an independent comparison of model reasoning.

The later optional 9B/12,288 profile produced 8/8 generated literal matches and 2/2 exact unknown abstentions on a five-document development probe. Context, instructions and citation retry also changed, so this is a postselected configuration result, not model-only or paper accuracy. All four compared runs failed the complete short-summary check. No held-out corpus or native Windows GPU study accompanies that probe.

Exact serialization prevents the observed avoidable evidence drop in the exercised requests. Sixty-four retained references do not imply 64 correct answers. Frozen raw outputs, distinct run identities and versioned source records separate historical failure, diagnostic development, the repair replay and later application changes. Semantic correctness, adversarial robustness and performance on representative personal libraries still require independent evidence.

8. Conclusion and Future Scope

8.1 Conclusion

LocalMind integrates approved-file indexing, source inspection, persistent SQLite memory, local inference and genuine MCP communication. Published context and grounding repairs are followed by tested greeting isolation, exact metadata routes and whole-record evidence identity. Source cards group repeated passages while preserving reference numbers. The result is an inspectable local workflow, with bounded actions and explicit limits rather than guaranteed factual assistance.

Historically, the original generators completed 0/24 positives per method after dropping evidence. A separate repaired replay retains references in 64/64 requests without errors. Single-pass AFVM is 7/24 with macro FVM 37.50%; agent AFVM is 3/24 with macro FVM 27.08%. These reused-question observations remain unchanged. The later 9B selection probe and latest functional checks have separate records and do not replace the manuscript experiment.

Latest checks pass 163 Linux and 200 Windows-copy tests on Linux, with six native skips. The optional 9B development configuration reaches 8/8 known literal checks and 2/2 unknown abstentions after selection; this is not general accuracy. Historical summaries retain only 10/24 values. Citation bounds do not establish entailment, and broader factual reliability, representative-library evaluation and native Windows GPU validation remain open work.

8.2 Future Scope

Exact serialized fitting, whole-record evidence identity, local social responses and exact catalogue routing are implemented. Further context work should test field-aware selection and preserve useful coverage when candidate windows shrink or a record cannot fit. Source cards show delivered references but cannot prove their semantic use. A held-out corpus is needed before claiming general factual reliability.

Task-aware evidence selection remains a priority. Query-aware windows now improve selection within returned passages, but relevant material can still lie outside retrieved chunks or bounded windows. Structured extraction and source-linked checks may reduce summary losses. Fair model comparisons should hold context, instructions and retry policy constant, include tasks requiring further reads, and report whether additional tools deliver missing facts.

Operational improvements include optional local OCR, opt-in watching, content-hash freshness, memory export and import, encrypted storage and systematic prompt-injection tests. Versioned context and grounding repairs are already published. Source changes require fresh native Windows validation; historical CI and Linux execution of the Windows copy cannot substitute for checks of changed code.

Real Windows 11 GPU runs, heterogeneous and multilingual files, larger archives and consented user studies remain necessary. Future experiments should distinguish model loading from warm execution and measure power and human verification effort. These proposed activities are separate from completed local routing, identity and context fixes, their regression checks and the limited synthetic development probe.

9. References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*. https://arxiv.org/abs/2005.11401

[2] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 6769–6781). https://doi.org/10.18653/v1/2020.emnlp-main.550

[3] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* (pp. 3982–3992). https://doi.org/10.18653/v1/D19-1410

[4] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval, 3*(4), 333–389. https://doi.org/10.1561/1500000019

[5] SQLite developers. (n.d.). *SQLite FTS5 extension*. Retrieved October 17, 2026, from https://sqlite.org/fts5.html

[6] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759). https://doi.org/10.1145/1571941.1572114

[7] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

[8] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2302.04761

[9] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as operating systems* [Preprint, revised 2024]. arXiv. https://doi.org/10.48550/arXiv.2310.08560

[10] Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence, 38*(17), 19724–19731. https://doi.org/10.1609/aaai.v38i17.29946

[11] Wang, W., Dong, L., Cheng, H., Liu, X., Yan, X., Gao, J., & Wei, F. (2023). Augmenting language models with long-term memory. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2306.07174

[12] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). *Generative agents: Interactive simulacra of human behavior* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2304.03442

[13] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2303.11366

[14] Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. https://doi.org/10.1145/3359591.3359737

[15] Model Context Protocol contributors. (2025). *Model Context Protocol specification* (Revision 2025-11-25). Retrieved October 17, 2026, from https://modelcontextprotocol.io/specification/2025-11-25

[16] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). *Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2302.12173

[17] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, 1*. https://arxiv.org/abs/2104.08663

[18] Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics* (pp. 2014–2037). https://doi.org/10.18653/v1/2023.eacl-main.148

[19] Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 150–158). https://doi.org/10.18653/v1/2024.eacl-demo.16

[20] Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6465–6488). https://doi.org/10.18653/v1/2023.emnlp-main.398

[21] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

[22] FastAPI contributors. (n.d.). FastAPI [Documentation]. Retrieved October 18, 2026, from https://fastapi.tiangolo.com/

[23] Uvicorn contributors. (n.d.). Uvicorn [Documentation]. Retrieved October 18, 2026, from https://uvicorn.dev/

[24] Ollama. (n.d.). Introduction. Ollama API documentation. Retrieved October 18, 2026, from https://docs.ollama.com/api/introduction

[25] Ollama. (n.d.). *FAQ*. Retrieved October 17, 2026, from https://docs.ollama.com/faq

[26] Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., … Qiu, Z. (2025). *Qwen3 technical report* [Technical report]. arXiv. https://doi.org/10.48550/arXiv.2505.09388

[27] Vera, H. S., Dua, S., Zhang, B., Salz, D., Mullins, R., Panyam, S. R., Smoot, S., Naim, I., Zou, J., Chen, F., Cer, D., Lisak, A., Choi, M., Gonzalez, L., Sanseviero, O., Cameron, G., Ballantyne, I., Black, K., Chen, K., … Seyedhosseini, M. (2025). *EmbeddingGemma: Powerful and lightweight text representations* [Technical report]. arXiv. https://doi.org/10.48550/arXiv.2509.20354

[28] SQLite developers. (2022, April 18). Isolation in SQLite. https://sqlite.org/isolation.html

[29] pypdf contributors. (n.d.). Extract text from a PDF. pypdf 6.0.0 documentation. Retrieved October 18, 2026, from https://pypdf.readthedocs.io/en/6.0.0/user/extract-text.html

[30] NumPy developers. (n.d.). numpy.linalg.norm. NumPy v2.2 manual. Retrieved October 18, 2026, from https://numpy.org/doc/2.2/reference/generated/numpy.linalg.norm.html

[31] Model Context Protocol contributors. (n.d.). MCP Python SDK [Version 1.x documentation]. Retrieved October 18, 2026, from https://py.sdk.modelcontextprotocol.io/v1/

[32] HTTPX contributors. (n.d.). Environment variables. Retrieved October 18, 2026, from https://www.python-httpx.org/environment_variables/

[33] TrijalBurman. (2026b). *Native Windows validation workflow* (Run 37470160711, commit 0e0bbf6f944bc6f2534f882c858ca4cdd22e6443) [Continuous integration execution]. GitHub Actions. https://github.com/TrijalBurman/MCP-Server/actions/runs/37470160711

[34] TrijalBurman. (2026a). *LocalMind local knowledge copilot—Linux edition* (Commit ddbbbb7be35a83c6ebc32706bb0aaf41ff379ff8) [Source code]. GitHub. https://github.com/TrijalBurman/MCP-Server/tree/ddbbbb7be35a83c6ebc32706bb0aaf41ff379ff8

Appendices

A. Reproducibility record

Frozen corpus, baseline, diagnostic and production repair replay remain under research/evaluation. The replay is 2026-10-17-production-context-fix-v1. Later model-selection records are in research/model-selection. Protocols, raw responses, hashes and verification records keep these development stages separate.

Published repair anchors are main 4a3533a and Windows b377ac6. Historical experiments use ddbbbb7 [34] and 0e0bbf6 [33]. The replay fitter SHA-256 remains 1db1e79b7eac3dba77b2afaa2e852db847b7ae2169d7f92327b750dbaa4cdc25. New routing and display changes retain separate source records.

Item	Recorded setting
Chat model	qwen3:4b-instruct-2507-q4_K_M
Embedding model	embeddinggemma:latest; 768 dimensions
Chat context / generation	8,192 / 1,536 tokens; temperature 0.25
Runtime	Python 3.14.4; Ollama 0.35.0
Hardware	Ryzen 7 250; RTX 5050 Laptop; nominal 16 GB RAM

Table 11: Historical reproduction settings

The table records the historical 4B experiment. Source defaults remain 4B/8,192; optional 9B/12,288 has separate probe records. Use new run IDs and preserve hashes. Native Windows CI run 37470160711 applies to its historical commit.

B. Demonstration acceptance cases

Use the supplied examples/knowledge folder for a compact demonstration. Connect it under Library, confirm two documents are indexed and open lighthouse-project.md. The reader should display the complete extracted text; this is the ground truth against which a generated answer is checked. The original files remain in the demonstration folder.

A suitable factual question is: What storage does the Lighthouse project use? The source states SQLite for extracted text, chunks, conversations and memories. A second question asks whether image-only PDFs work: the supplied note says scanned PDFs need OCR, which the application does not provide. Do not substitute a speculative answer if the model abstains.

Check	Expected observation / recovery
Library reader	Source text opens; show it if inference stalls
Project creation	Named workspace appears in Projects
Explicit memory	Saved project fact survives navigation / new chat
Source-linked answer	Compare response with source reader

Check	Expected observation / recovery
Optional summary	Verify draft; use saved rehearsal if slow

Table 12: Demonstration checks and recovery

The normal and demo apps share one local model service while retaining separate data directories. Live metadata checks found seven indexed files, one exact api.md match and no match for an absent filename; these are indexed-scope observations, not a drive search. Complete greetings use no retrieval or inference. Label saved outputs as rehearsal evidence and verify answers against the reader during the short live demonstration.

C. Academic integrity and required reports

Institutional submission records may include an AIC form, a similarity report and an AI plagiarism report. Those institutional records have not been supplied for this submission package. The table below records their status; it is not a replacement certificate or an invented score. Obtain the institution's current forms and reports, review them with the guide and attach them before formal submission.

AI assistance was used for software development, synthetic corpus construction, evaluation support and manuscript/report drafting. Authors remain responsible for checking implementation claims, numerical results, references, institutional declarations and presentation statements. AI assistance does not constitute independent peer review. The synthetic dataset is an explicit experimental artifact rather than naturally collected user behaviour.

Required attachment	Current status
AIC form	Institution-issued form not provided
Similarity report	Official report and score not provided
AI plagiarism report	Official report and score not provided
Faculty signatures	Unsigned; to be completed by authorized faculty
Student declaration	For authors' review and signature

Table 13: Mandatory attachment register

No signature, approval date, originality percentage or plagiarism outcome has been generated. If institutional policy requires additional disclosure language, the authors should replace this statement with the approved wording after reviewing the underlying evidence.

D. Submission and evidence checklist

The companion IEEE-format research paper acknowledges Dr. Sonali Kothari and lists the four student authors. It is a prepared manuscript; no acceptance, publication, DOI or conference presentation is claimed. The template's optional published-paper page should be attached only after an actual publication exists. Industry, competition and achievement certificates are likewise optional evidence, not created records.

Before printing, verify all names and PRNs, sign the declaration after reading it, obtain authorized faculty signatures and enter the submission date. The report identifies the programme as Bachelor of Computer Science Engineering and the academic year as 23–27. Keep the editable file and its matching verified PDF together so that any later edit can be repaginated and the contents list updated.

Evidence	**Location / handling**
Implementation	Versioned source and dated verification records
Research evidence	research/evaluation and accompanying archive
Paper	IEEE source and compiled PDF
Presentation preparation	Four-member 10-minute script and demo guide
Local data	Back up SQLite and project PLAN.md with app stopped

Table 14: Project evidence and preservation

This report separates the frozen baseline, diagnostic, recorded repair replay and later implementation checks. The earlier repairs are published; latest source and document refreshes retain their own release records. Preserve prior archives and checksums as historical evidence rather than rewriting them to match current behavior. Re-render later edits and verify the sample-based layout and current page references before submission.